Data Analysis Foundations — I

Lecture Notes · The Data Matrix, Attribute Types, and the Algebraic, Geometric & Probabilistic Views of Data

DS605 · Fundamentals of Machine Learning · Lectures 02–03

Arpit Rana

Running example throughout: the ShopKart e-commerce dataset

Before We Open a Single Formula

WHY DOES ANY OF THIS MATTER

Computers make "similarity" decisions constantly, even though a computer has no built-in sense of what two things have in common. **Netflix** recommends movies similar to ones you liked. **Google Photos** groups photos of the same face together. **Amazon — and our own ShopKart —** recommends products similar to what you have browsed. **ChatGPT** turns text into vectors before it can "understand" it. **Self-driving cars** cluster LiDAR points to recognize objects. In every one of these systems, "similar" has been turned into something a machine can compute.

The Question This Lecture Answers

How does a computer know that two things are similar? Everything in this lecture — data matrices, vectors, geometry, rank, probability — is machinery built to answer exactly this one question. Nothing here is isolated mathematics. The route we take is:

```
Vectors → Distance → Probability → Statistics → Similarity
```

Meet ShopKart

ShopKart is the same fictional e-commerce platform introduced in Lecture 01 — 2.4 million customers in total. For today's hands-on mathematics, we work with a random sample of 50,000 of them: small enough to compute with by hand or on a laptop, large enough to be realistic. One dataset, one story, threaded across four lectures, so every new idea builds on one you already understand.

Lecture	Topic	What happens to the data
02	Attributes · Vectors · Geometry	Customers become points in space.
03	Probability	Customer behavior becomes a random variable.
04	Mean · Variance · Boxplots	Summarising customer spend.
05	Distance Measures	Euclidean, cosine, and Mahalanobis distance, in practice.

> **KEEP IN MIND** We return to the exact same customers in every lecture — 2.4 million total customers, 50,000-customer working sample. That distinction between the full population and our working sample matters, and we make it precise in Part 4.

Part 1 — The Data Matrix & Attribute Types

This part covers the vocabulary every machine learning paper assumes you already know: instances, attributes, dimensionality, and the different flavors an attribute can come in.

The Data Matrix

- The number of instances ***n*** is called the **size** of the data; the number of attributes ***d*** is the **dimensionality** of the data.
- Analyzing a single attribute is called **univariate** analysis.
- Analyzing two attributes together is called **bivariate** analysis.
- Analyzing more than two attributes together is called **multivariate** analysis.

SHOPKART, CONCRETELY

$$D \in \mathbb{R}^{n \times d}, \quad n = 50{,}000 \text{ customers (working sample)}, \quad d = 12 \text{ attributes}$$

D itself is 50,000 rows by 12 columns. We can only ever show a corner of it — rows run off the bottom, columns run off the side, because a matrix this size cannot fit on a page, or in your head. Everything that follows is machinery for reasoning about the whole matrix without ever looking at it in full.

Dataset Glossary (ShopKart, Column by Column)

Attribute	Meaning
customer_id	Unique identifier for the row (not used in analysis)
age	Customer's age in years
annual_income	Reported yearly income (₹)
city	City the customer ships to
membership_tier	Bronze / Silver / Gold / Platinum
preferred_category	Electronics / Fashion / Grocery / Books
num_orders_last_year	Count of orders placed in 12 months
avg_order_value	Average order size (₹)
is_prime_member	Subscribed to ShopKart Prime? (Yes/No)
rating_given	Average product rating the customer gives (1–5)
days_since_last_purchase	Recency of last order
device_type	Mobile / Desktop / Tablet

Data Objects and Attributes

A **data object** represents an entity — one *row* in the database. It is also called an entity, sample, example, instance, data point, domain point, feature vector, tuple, or simply an object. Row 4,213 of our data matrix is not just "a row" — it *is* a specific ShopKart customer: 34 years old, Gold tier, with an average order of ₹ 1,850.

An **attribute** is a data field representing a characteristic of a data object — a *column*. It is also called a property, feature, dimension, variable, or field. The set of attributes used to describe a customer — {age, income, orders, avg_order_value, ...} — is called an **attribute vector**. Every customer is described by the same 12 attributes.

Why Vectors Matter

Turning a row into a vector is not bookkeeping — it is what lets a computer compare things it has no built-in understanding of: whether two ShopKart customers are similar enough that one's purchases predict the other's, whether two product reviews are talking about the same issue, whether two product photos show the same item, how closely related two genetic sequences are, or whether two sentences mean the same thing to a language model. Everything in Parts 2–4 builds the machinery to answer exactly these kinds of questions for ShopKart customers.

> **WHERE THIS IS HEADING** Every modern AI student meets "embedding" before they meet "vector" — they are the same idea. A sentence goes into an embedding model and comes out as a 768-dimensional vector, ready for similarity search. A word embedding, a customer row, and a product-image vector are the same mathematical object: a point in $\mathbb{R}^d$. This entire lecture explains why embeddings work.

Structured, Unstructured, Semi-Structured

Kind	Description	ShopKart example
Structured	Small, fixed set of attributes	The customer profile table: age, income, tier, orders...
Unstructured	No explicit structure	Free-text product reviews: "battery dies after a day, otherwise great"
Semi-structured	A mixture of the two	Product tags (Electronics, Wireless, Refurbished) attached to a free-text description

Attribute Types

An attribute can be classified along two independent axes. The first axis is qualitative versus quantitative:

Category	Definition	ShopKart example
Qualitative	No true size or quantity — words representing categories	preferred_category
Quantitative	A quantitative measurement	avg_order_value

Qualitative attributes are further split into nominal and ordinal; quantitative attributes are further split into interval-scaled and ratio-scaled. Independent of that split, any attribute is also discrete (finite or countable) or continuous (any real value).

Nominal Attributes (a.k.a. Categorical)

Values are symbols or names representing a category, with no inherent order. Any two categories can be freely swapped, and the attribute's meaning is completely unchanged.

> **SHOPKART EXAMPLES** preferred_category (Electronics, Fashion, Grocery, Books) · city (Mumbai, Pune, Delhi …) · device_type (Mobile, Desktop, Tablet)

Binary Attributes

Binary attributes are nominal attributes with only two states (0/1, true/false). They come in two flavors:

- **Symmetric** — is_prime_member: "Yes" and "No" are equally important outcomes.
- **Asymmetric** — fraud_flag: "fraud detected" carries far more weight than "not fraud."

Ordinal Attributes

Ordinal attributes are nominal attributes whose values have a meaningful order, but the gap between successive values is not well defined. membership_tier runs Bronze < Silver < Gold < Platinum, and rating_given runs 1★…5★ — the order is certain, but whether the "distance" from Silver to Gold equals the distance from Gold to Platinum is not something the data tells us.

Numeric Attributes: Interval vs. Ratio

Scale	Defining property	ShopKart examples
Ratio-scaled	Has an inherent zero-point; never falls below zero	age, annual_income, num_orders_last_year, days_since_last_purchase
Interval-scaled	No true zero; values below zero are meaningful	satisfaction_score on a −5…+5 survey scale · signup_year (a calendar value)

The distinction is the presence of a true zero. ₹0 income is a true absence of income, so annual_income is ratio-scaled. A satisfaction score of 0 on a −5…+5 scale is just a neutral point, not "zero satisfaction" — so it is interval-scaled.

Quick Self-Check

Before moving on, decide whether each ShopKart attribute below is interval-scaled or ratio-scaled — then check your reasoning against the answer.

ShopKart Attribute	Answer
Customer age	Ratio — 0 is a true bound
Annual income (₹)	Ratio — ₹0 income is meaningful
Signup year (e.g., 2021)	Interval — “year 0” isn't a meaningful absence
Satisfaction score (−5 to +5)	Interval — can go negative, no true zero
Days since last purchase	Ratio — 0 days means "purchased today"
Order value (₹)	Ratio — ₹0 order is a true zero

Part 2 — Algebraic & Geometric View of Data

How does ShopKart compare two customers? By turning every row into a point in space, and every question about "similarity" into a question about geometry.

Every Row a Point, Every Customer a Vector

Customer #4213's row in the data matrix — age = 34, income = ₹9,20,000, orders = 18, avg_order_value = ₹1,850, … — is the exact same customer viewed as a point $x_i \in \mathbb{R}^{12}$: a specific coordinate in 12-dimensional space. Once two customers are points, "how similar are they?" becomes "how close are these two points?" — a question geometry already knows how to answer.

Points, Vectors, and the Coordinate System

If all the d attributes in the data matrix D are numeric, each row is a d-dimensional point — equivalently, a d-dimensional column vector (all vectors are column vectors by default):

$$x_i = (x_{i1}, x_{i2}, \ldots, x_{id})^T \in \mathbb{R}^d$$

> **SHOPKART** x_i = (age i, income i, orders i, …)^T — customer i's full attribute vector.

The d-dimensional Cartesian space is defined by d standard basis (unit) vectors, one per attribute. The j-th standard basis vector e_j has a 1 in position j and 0 elsewhere:

$$x_i = \sum_{j=1..d} x_{ij} \cdot e_j$$

> **SHOPKART** In the simplified 2-D "income vs. orders" plane, $e_1 = (1,0)^T$ points along income and $e_2 = (0,1)^T$ points along orders — every customer is a weighted combination of these two directions.

The Data Matrix Read Two Ways

Each numeric column is also a vector — this time in $\mathbb{R}^n$. So D can be read two ways: as n row vectors $x_i^T \in \mathbb{R}^d$ (50,000 customers, each a point in 12-D attribute space), or as d column vectors $X_j \in \mathbb{R}^n$ ("avg_order_value across all 50,000 customers" is itself one long vector). The table below highlights both directions in the same excerpt of D.

	age	income	orders/yr	avg_order_value	rating	days_since
$x_1^T \rightarrow$	28	₹4,80,000	22	₹1,120	4.2	6
$x_2^T \rightarrow$	34	₹9,20,000	18	₹1,850	4.6	2
$x_3^T \rightarrow$	51	₹3,10,000	9	₹640	3.8	19

The highlighted row is customer x_2 read across — a row vector $x_2^T \in \mathbb{R}^6$. The avg_order_value column read down is a column vector $X_4 \in \mathbb{R}$ 50,000. Same matrix, two completely different vectors, depending on which direction you read it.

Dot Product

$$a^T b = \Sigma_{k=1..m} \; a_k b_k$$

PYTHON — TRY IT YOURSELF

```
import numpy as np
a = np.array([2.0, 3.0])   # customer A: (spend'000, orders)
b = np.array([7.0, 6.0])   # customer B
dot = np.dot(a, b)         # exactly a^T b
print("a . b =", dot)      # -> 32.0
```

Length (Euclidean Norm)

$$\|a\| = \sqrt{(a^T a)} = \sqrt{(\Sigma_{k=1..m} a_k^2)} \qquad \text{unit vector: } u = a / \|a\|, \; \|u\| = 1$$

The norm is a vector's length — its distance from the origin. Dividing a vector by its own norm produces a unit vector: same direction, length exactly 1.

Customer	(electronics, grocery) ₹'000/mo	$\|\cdot\|$	Unit vector
A — light shopper	(2, 8)	$\sqrt{68} \approx 8.25$	(0.243, 0.970)
B — heavy shopper, same pattern	(6, 24)	$\sqrt{612} \approx 24.74$	(0.243, 0.970)

WORKED EXAMPLE B spends exactly 3× what A spends in both categories — the same 1-electronics-to-4-grocery habit, just scaled up. Their raw vectors and norms are completely different (8.25 vs. 24.74), but once each is divided by its own norm, the unit vectors are identical: (0.243, 0.970). Normalizing a customer's spending vector removes the effect of "how much they buy overall" and retains only their spending pattern across categories — exactly what recommender systems want.

Euclidean Distance

$$\|a - b\| = \sqrt{(\Sigma\ k=1..m\ (a\ k - b\ k)^2)}$$

The length of a vector is just its distance from the zero vector: $\|a\| = \|a - 0\|$.

PYTHON — TRY IT YOURSELF

```
import numpy as np
from scipy.spatial import distance
a = np.array([2.0, 3.0])
b = np.array([7.0, 6.0])
d = distance.euclidean(a, b)   # ||a - b||
print("distance(a, b) =", round(d, 3))   # -> 5.831
```

Angle & Cosine Similarity

$$\cos\ \theta = a\ ^{T}b\ /\ (\|a\|\ \|b\|) = u\ a^{T}\ u\ b$$

Since $\theta \in [0°, 180°]$, cosine similarity ranges from +1 (identical direction) to −1 (opposite direction). Two customers can be far apart in raw distance (different overall spend levels) yet point in almost the same direction (a similar shopping pattern) — distance and cosine similarity move independently.

PYTHON — TRY IT YOURSELF

```
import numpy as np
a = np.array([2.0, 3.0])
b = np.array([7.0, 6.0])
cos_theta = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print("cosine similarity =", round(cos_theta, 3))   # -> 0.963

# In practice, you would typically just call:
# from sklearn. metrics.pairwise import cosine_similarity
```

WHERE THIS IS USED Cosine similarity between vectors is quite possibly the single most-used equation in modern AI: Google Search, sentence transformers, ChatGPT embeddings, retrieval-augmented generation (RAG), semantic search, and recommendation systems all rest on it.

Orthogonality

$$a^\top b = 0 \iff \cos\theta = 0 \iff \theta = 90°$$

Two vectors are orthogonal when their dot product is zero — geometrically, a 90° angle. We say they have no similarity. A customer who buys only groceries and one who buys only electronics might have near-orthogonal category-spend vectors.

Orthogonal Projection

We can decompose b into a part parallel to a and a part perpendicular to it: $b = b_{\parallel} + b_{\perp} = p + r$, where p is the "shadow" b would cast onto the line through a, and r is what is left over, at a right angle to a by construction — that is exactly what forcing $p^\top r = 0$ buys us.

$$p = c \cdot a, \quad r = b - c \cdot a, \quad p^\top r = 0 \implies c = a^\top b \,/\, a^\top a$$

This is useful whenever we need to re-express a point after changing basis vectors — a preview of what PCA does later in Part 3.

From Dot Product, to Projection, to Similarity

Three ideas just introduced are really one idea, seen at different amounts of "scaling removed." The dot product mixes together how far apart two vectors point and how long each one is. Divide once by $\|a\|$, and you get the scalar projection — how far along a's direction b actually reaches, still in b's original units. Divide again by $\|b\|$ and both lengths cancel: all that is left is $\cos\theta$, pure direction, no scale at all.

$$\text{Dot product } (a^\top b) \;\to\; \div\|a\| \;\to\; \text{Scalar projection } (\|b\|\cos\theta) \;\to\; \div\|b\| \;\to\; \text{Cosine similarity } (\cos\theta)$$

Quantity	Formula	a=(2,3), b=(7,6)
Dot product	$a^T b$	$2 \cdot 7 + 3 \cdot 6 = 32$
Norms	$\|a\|$, $\|b\|$	$\sqrt{13} \approx 3.606$, $\sqrt{85} \approx 9.220$
Scalar projection	$a^T b / \|a\|$	$32 / 3.606 \approx 8.875$
Cosine similarity	$a^T b / (\|a\|\|b\|)$	$32 / (3.606 \times 9.220) \approx 0.963$

READING THE RESULT $\cos\theta \approx 0.963$ is close to 1 — these two customers shop in almost the same pattern, even though b spends more overall ($\|b\| \approx 9.22$ vs. $\|a\| \approx 3.61$). Distance would call them far apart; cosine similarity correctly calls them alike.

Mean & Total Variance of D

$$\text{mean}(D) = (1/n) \sum_{i=1..n} x_i \qquad \text{var}(D) = (1/n) \sum_{i=1..n} \|x_i - \text{mean}(D)\|^2$$

mean(D) is the "average ShopKart customer." var(D) is the average squared distance from that average customer.

PYTHON — TRY IT YOURSELF

```
import numpy as np
# excerpt of D: age, income, orders/yr, avg_order_value
D =
np.array([[28,480000,22,1120],[34,920000,18,1850],[51,310000,9,640],[29,340000,11,710]])
print("mean(D) =", np.mean(D, axis=0))
print("std(D)  =", np.std(D, axis=0).round(2))
```

Centered Data Matrix

We often shift the data so the mean sits at the origin:

$$\hat{D} = D - 1 \cdot \text{mean}(D)^T$$

Shifting the origin does not change the shape of the point cloud — only where it sits. Before you move on, think it through: what is the mean of the centered data $\hat{D}$? Does centering change the total variance var(D)? (Worth discussing before advancing.)

Part 3 — Linear Independence & Rank

Not every attribute adds new information. How many genuinely independent dimensions does ShopKart's data actually have?

Linear Combinations & Span

$$c_1v_1 + c_2v_2 + \ldots + c_k v_k, \quad c_i \in \mathbb{R}$$

The set of all such combinations is the span, span($v_1,\ldots,v_k$) — itself a subspace of $\mathbb{R}^m$. If it equals all of $\mathbb{R}^m$, the vectors form a spanning set.

When One Attribute Is "Hiding" Inside Others

A simple, non-ShopKart intuition first: BMI is approximately derived from height and weight, so among {Height, Weight, BMI}, BMI has rank less than 3. Redundant attributes shrink the true dimensionality — even when the relationship is only approximate, as with BMI.

Total_Bill Is Not an Independent Attribute

Now bring the idea back to ShopKart — exactly, not approximately. Total_Bill is completely determined by three other columns:

$$\text{Total_Bill} = \text{Subtotal} - \text{Discount} + \text{Shipping_Fee}$$

WORKED EXAMPLE Subtotal = ₹2000, Discount = ₹200, Shipping = ₹50 $\Rightarrow$ Total_Bill = ₹1850, exactly, every time — no new information is added by keeping this fourth column. Because Total_Bill is an exact linear combination of the other three, these four columns together have rank ≤ 3: one dimension is redundant.

Row Space & Column Space

$$\text{col}(D) = \text{span}(X_1,\ldots,X_d) \subseteq \mathbb{R}^n \qquad \text{row}(D) = \text{span}(x_1,\ldots,x_n) \subseteq \mathbb{R}^d = \text{col}(D^T)$$

col(D), the column space, is all combinations of the 12 ShopKart attributes. row(D), the row space, is all combinations of the 50,000 customer vectors.

Linear Independence

$$c_1 v_1 + c_2 v_2 + \ldots + c_k v_k = 0 \quad \Longrightarrow \quad c_1 = c_2 = \ldots = c_k = 0$$

If some vector in the set can be written as a combination of the others, the set is linearly dependent — exactly what happened with Total_Bill above.

Basis & Dimension

A basis for subspace S is a minimal, linearly independent set of vectors that spans S. If the vectors are mutually orthogonal, it is an orthogonal basis; if they are also unit length, it is an orthonormal basis (the standard basis is one example).

$$\dim(S) = \text{number of vectors in any basis for } S, \quad \dim(S) \leq m \text{ for } S \subseteq \mathbb{R}^m$$

Rank

$$\text{rank}(D) \leq \min(n, d)$$

- Data may appear to live in d dimensions — its extrinsic dimensionality.
- But if rank(D) < d, it actually lives in a lower-dimensional subspace — the intrinsic dimensionality.
- Dimensionality reduction approximates $D \in \mathbb{R}^{n \times d}$ with $D' \in \mathbb{R}^{n \times k}$, $k \ll d$, where k reflects the true intrinsic dimensionality.

Rank Is the Doorway to Modern ML

Rank/intrinsic dimensionality is the underlying principle behind PCA, autoencoders, latent spaces, embeddings, and compression. PCA can reduce ShopKart's 12 attributes to 2 "shopping persona" axes for a scatter plot; an autoencoder compresses a customer profile into a small code and reconstructs it; every customer, product, and review can become a point in the same learned space.

PYTHON — TRY IT YOURSELF

```
import numpy as np
# columns: Subtotal, Discount, Shipping, Total_Bill
# Total_Bill = Subtotal - Discount + Shipping  (exactly, every row)
D = np.array([
    [2000, 200, 50, 1850],
    [3000, 300, 80, 2780],
    [1500, 100, 30, 1430],
```

```
    [4000, 500, 60, 3560],
])
print("rank(D) =", np.linalg.matrix_rank(D))
# -> 3  (< 4 columns: Total_Bill adds no new information)

# Real-world dimensionality reduction, one line:
# from sklearn.decomposition import PCA
# PCA(n_components=2).fit_transform(D)
```

Part 4 — Probabilistic View of Data

Everything in machine learning assumes data is generated from probability distributions. This one idea underlies almost every algorithm in this course.

"Data Comes From a Distribution"

Six seemingly different techniques are all natural consequences of one assumption — that data is generated by a probability distribution:

- **Regression** — predict order value assuming it is drawn from a distribution around a true trend.
- **Classification** — decide Prime vs. non-Prime; each class has its own distribution of behavior.
- **Naive Bayes** — classify reviews as spam using independent per-word probabilities.
- **Gaussian Mixture** — discover hidden customer segments as a mixture of bell curves.
- **Diffusion Models** — generate product images by reversing a noise distribution.
- **Bayesian Learning** — update belief about a customer's taste as new orders arrive.

Random Variable

A random variable X is a function $X: \Omega \to \mathbb{R}$, mapping each outcome of an experiment (the sample space Ω) to a real number. X is discrete if it takes finitely or countably many values, and continuous if it can take any value in a range.

> **SHOPKART** Let X = "monthly spend of a randomly chosen ShopKart customer." Ω is the set of all customers; X(customer) is their spend, a real number.

Discrete vs. Continuous Random Variables

Kind	ShopKart example
Discrete	num_orders_last_year — can only be 0, 1, 2, 3, …
Continuous	avg_order_value — can be any real number, e.g., ₹1,847.23

Probability Mass Function (for discrete X)

$$f(x) = P(X = x), \quad f(x) \geq 0, \quad \Sigma_x f(x) = 1$$

> **SHOPKART** P(membership_tier = Gold) — probability mass concentrated at each discrete tier. Bernoulli and Binomial distributions live here.

Probability Density Function (for continuous X)

$$P(a \le X \le b) = \int_a^b f(x)\,dx$$

Since P(X = x) = 0 for any single value, we measure probability over intervals — the density f(x) is probability per unit distance, not a probability itself.

> **SHOPKART** The density of avg_order_value near ₹2,000 tells us how "crowded" that region is — not the exact probability of ₹2,000.00.

Cumulative Distribution Function

$$F(x) = P(X \le x), \quad F: \mathbb{R} \to [0, 1]$$

> **SHOPKART** What fraction of ShopKart customers spend ≤ ₹2,000 on average? Read it directly off F(2000).

Multivariate Random Variable

$$X = (X_1, X_2, \ldots, X_d)^T : \Omega \to \mathbb{R}^d$$

If every outcome in the sample space is itself a numeric vector, we can treat every row of the ShopKart data matrix as one draw of a vector random variable — the exact same x i from Part 2, now viewed as a random draw rather than a fixed geometric point.

Population, Random Sample & Statistic

Term	Meaning
Population	All 2.4 million ShopKart customers, past, present and future — too large or expensive to observe fully.
Random sample	Our 50,000-customer working sample, drawn from that population: i.i.d. draws $S_1,\ldots,S_n$ from X.
Statistic	$\hat{\theta} = f(S_1,\ldots,S_m)$ — a function of the sample used to estimate a population parameter θ (a "point estimate").

$$\text{Joint density of an i.i.d. sample:} \quad f(S_1,\ldots,S_n) = \prod_{i=1}^{n} f_X(S_i)$$

Multivariate Sample

Our n customer vectors x_i form a multivariate random sample drawn from X = (X_1,..., X_d), which is independent and identically distributed. A common simplifying assumption is that the d attributes are statistically independent — but almost never identically distributed, since income and orders clearly do not follow the same distribution.

The Normal (Gaussian) Distribution

Many physically observed variables are approximately normal, and the sampling distribution of the mean of any distribution tends toward normal — the Central Limit Theorem.

Univariate Normal Distribution

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2} \right)$$

SHOPKART If avg_order_value is roughly bell-shaped around μ = ₹1,850, the density falls off exponentially as we move away — fastest near the peak, where $f(\mu) = 1/\sqrt{(2\pi\sigma^2)}$.

PYTHON — TRY IT YOURSELF

```
from scipy.stats import norm

mu, sigma = 1850, 400   # avg_order_value: mean, std (₹)
x = 2000
density = norm.pdf(x, loc=mu, scale=sigma)
print(f"f({x}) = {density:.6f}")
```

Multivariate Normal Distribution

$$f(x) = \frac{1}{((2\pi)^{d/2} |\Sigma|^{1/2})} \cdot \exp\left(-\tfrac{1}{2} (x-\mu)^{T} \Sigma^{-1} (x-\mu) \right)$$

Now $\mu \in \mathbb{R}^d$ is the mean vector (the "average customer" from Part 2), and Σ is the d × d covariance matrix, capturing how attributes move together — e.g., income and order value.

Mahalanobis Distance

Remember the Euclidean distance between two ShopKart customers from Part 2? Mahalanobis distance is the upgrade that accounts for correlated attributes.

$$D_M(x) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$$

THE KEY INSIGHT Set $\Sigma = I$ and it collapses exactly to Euclidean distance: $\sqrt{(x-\mu)^T(x-\mu)}$. Euclidean distance ignores covariance; Mahalanobis distance uses it — e.g. it knows income and order value tend to rise together, so it is not fooled by that correlation.

PYTHON — TRY IT YOURSELF

```
import numpy as np
from scipy.spatial.distance import mahalanobis

mean = np.array([1850, 18])            # avg_order_value, orders/yr
cov  = np.array([[400**2, 3000],
                 [3000,      16]])     # covariance: these attrs move together
x = np.array([3000, 5])                # an unusual customer: big basket, few orders

d_mahal = mahalanobis(x, mean, np.linalg.inv(cov))
d_eucl  = np.linalg.norm(x - mean)
print("Mahalanobis distance =", round(d_mahal, 3))
print("Euclidean distance   =", round(d_eucl, 3))
```

Geometry of the Multivariate Normal

Σ is symmetric positive semi-definite, so it has orthonormal eigenvectors u_i and eigenvalues $\lambda_1 \geq \ldots \geq \lambda_d \geq 0$ with $\Sigma u_i = \lambda_i u_i$:

$$\Sigma = U \Lambda U^T$$

- This is exactly a change of basis — the same idea from Part 3's basis and rank discussion.
- In the new (eigenvector) coordinates, the distribution is axis-aligned: no rotation, only per-axis scaling by λ_i.
- Shift comes from $\mu \neq 0$, scale comes from variances, and rotation comes from covariances.

WHERE THIS IS USED Fraud and anomaly detection, Gaussian mixture clustering, Naive Bayes spam filters, Bayesian A/B testing, diffusion models, and risk scoring all draw on the probabilistic view built in this part.

Wrap-Up — From Formulas to One Line of Code

Every equation covered today is exactly one Python function call.

Concept	ShopKart question	Python
Mean	Average order value across all customers	np.mean(x)
Std. deviation	How spread out are order values?	np.std(x)
Euclidean distance	How different are the two customers?	scipy.spatial.distance.euclidean(a,b)
Cosine similarity	How similar is their shopping pattern?	sklearn.metrics.pairwise.cosine_similarity
Rank	How many independent shopping factors exist?	np.linalg.matrix_rank(D)
Mahalanobis distance	Is this customer's behavior unusual?	scipy.spatial.distance.mahalanobis

Try the Whole Lecture in Code

Mean, standard deviation, Euclidean distance, cosine similarity, rank, and Mahalanobis distance — run it, then edit the numbers and re-run.

PYTHON — TRY IT YOURSELF

```
import numpy as np
from scipy.spatial import distance
from scipy.spatial.distance import mahalanobis

# age, income, orders/yr, avg_order_value — 4 ShopKart customers
D = np.array([
    [28, 480000, 22, 1120],
    [34, 920000, 18, 1850],
    [51, 310000,  9,  640],
    [29, 340000, 11,  710],
])
a, b = D[0], D[1]   # compare two customers directly

print("mean(avg_order_value)  =", np.mean(D[:, 3]))
print("std(avg_order_value)   =", round(np.std(D[:, 3]), 2))
print("euclidean(a, b)        =", round(distance.euclidean(a, b), 2))

cos = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print("cosine similarity(a,b) =", round(cos, 4))
print("rank(D)                =", np.linalg.matrix_rank(D))

mean_vec = np.mean(D, axis=0)
cov = np.cov(D, rowvar=False) + np.eye(4) * 1e-6   # tiny ridge so it is invertible
d_mahal = mahalanobis(a, mean_vec, np.linalg.inv(cov))
print("mahalanobis(a, mean)   =", round(d_mahal, 3))
```

We Answered the Opening Question

"How does a computer know two things are similar?" — it turns them into vectors, measures distance or angle between them, and often assumes both were generated by an underlying probability distribution. Every future lecture is a variation on this theme:

```
Vectors → Distance → Probability → Statistics → Similarity
```

Coming Up — Lecture 04

> **NEXT LECTURE** Data Analysis Foundations II: Measures of Central Tendency & Dispersion — mean, variance, and boxplots, computed on the very same ShopKart customers met today.